

Rapport de projet : Application de Reconnaissance Faciale

Introduction

Ce projet est une application interactive qui permet de reconnaître des visages grâce à Python et Streamlit. L'idée est de pouvoir ajouter des visages dans une base de données et de les reconnaître ensuite via une webcam ou une vidéo. C'est une façon simple et amusante de montrer comment fonctionne la reconnaissance faciale.

Mode d'emploi de l'application

Pour installer les dépendances :

```
pip install -r requirements.txt
```

Lancement de l'application :

```
streamlit run app.py
```

Ajouter des visages connus :

1. Chargez une image de visage avec **"Charger une image de visage"**.
2. Écrivez le nom de la personne dans le champ texte.
3. Cliquez sur **"Ajouter à la base des visages connus"** pour enregistrer ce visage.
4. Les visages ajoutés s'affichent dans une liste appelée **"Visages connus configurés"**.
5. Si vous voulez tout effacer, utilisez **"Vider la base des visages connus"**.

Reconnaître des visages :

1. Choisissez une source vidéo : Webcam ou fichier vidéo.
 2. Si vous utilisez un fichier, chargez-le avec le bouton prévu.
 3. Cliquez sur **"Démarrer la reconnaissance"** pour lancer l'analyse.
 4. Les visages détectés apparaîtront avec leur nom ou **"Inconnu"**.
 5. Cliquez sur **"Arrêter la reconnaissance"** pour arrêter.
-

Résumé du code

Librairies principales :

- **Streamlit** : Pour créer une interface utilisateur simple et interactive.
- **face_recognition** : Pour détecter et reconnaître les visages.

- **OpenCV** : Pour traiter les images et afficher des cadres autour des visages.
- **NumPy** : Pour gérer les données sous forme de tableaux.
- **Pillow** : Pour ouvrir et manipuler les images.

face_recognition :

La librairie **face_recognition** repose sur le modèle de réseau neuronal profond d'OpenFace, qui utilise une architecture appelée **ResNet**. Voici comment elle fonctionne :

- **Localisation des visages** : La fonction `face_recognition.face_locations()` identifie les zones de l'image contenant un visage. Cela se fait grâce à un détecteur de visages basé sur un CNN.
 - **Encodage des visages** : La fonction `face_recognition.face_encodings()` extrait des caractéristiques uniques de chaque visage sous forme de vecteurs **embeddings** qui sont des représentations mathématiques des traits faciaux.
 - **Comparaison des visages** :
 - `face_recognition.compare_faces()` compare les vecteurs des visages détectés avec ceux des visages connus pour déterminer s'ils correspondent.
 - `face_recognition.face_distance()` calcule une distance entre les vecteurs, indiquant la similarité (plus la distance est faible, plus les visages sont proches).
 - **Simplicité et performance** : J'ai utilisé cette librairie car elle est simple à utiliser tout en offrant des résultats précis et rapides.
-

Comment ça marche :

Stockage des données :

- Les visages connus (encodages et noms) sont sauvegardés dans une session temporaire grâce à `st.session_state`.

Ajout de visages :

- Une image est chargée, convertie en tableau de données, et analysée pour extraire un encodage facial.
- Si l'image contient un visage, l'encodage et le nom sont ajoutés à la base.

Reconnaissance faciale :

- Les images de la webcam ou de la vidéo sont analysées en direct.
 - Les visages sont localisés et leurs encodages comparés à ceux de la base.
 - Si un visage est reconnu, le nom est labellisé, sinon il est marqué comme **"Inconnu"**.
-

Modèle utilisé :

Le modèle de reconnaissance de **face_recognition** repose sur des réseaux neuronaux. Il est efficace et facile à intégrer pour des projets comme celui-ci.

Conclusion

Cette application montre de façon simple comment on peut utiliser la reconnaissance faciale dans un projet. Même si elle est basique, elle peut être le point de départ pour des idées plus poussées comme par exemple dans des systèmes de sécurité. Cela a été pour moi une bonne expérience et une bonne façon de mettre en pratique les concepts théoriques sur les CNN vus en cours.

Réalisé par Michael Diop Rogandji